NOTES:
Look into grammars in which the derivations are derived by probabilities assigned to each production rule. This won't be super helpful yet, but could be much stronger for applications in fields.
Utilize gene bank optimization through schema to break out of local optima. In reproduction. Maybe mutate one child, and gene bank optimize the other?

[CS 4731]Paper 2

1. ABSTRACT

2. INTRODUCTION

The problem is that it gets stuck in an overgeneralized grammar and can't leave. This could be a problem in the fitness function, or in recombination. The main question this breaks into is given only positive examples, how does the algorithm bias towards the grammar we want considering an infinite amount of grammars also include the dataset.

Data for higher weighting on precision
Trial 1 (seed 42): best=103.259011211, recall=0.032000, precision=1.000000, rules=4
Trial 2 (seed 43): best=179.339215686, recall=1.000000, precision=0.980392, rules=7
Trial 3 (seed 44): best=103.259011211, recall=0.032000, precision=1.000000, rules=4
Trial 4 (seed 45): best=103.259011211, recall=0.032000, precision=1.000000, rules=4
Trial 5 (seed 46): best=179.339215686, recall=1.000000, precision=0.980392, rules=7

Nonterminals set to 10
Five-trial summary:
Trial 1 (seed 42): best=179.731372549, recall=1.000000, precision=0.980392, rules=7
Trial 2 (seed 43): best=179.731372549, recall=1.000000, precision=0.980392, rules=7
Trial 3 (seed 44): best=107.35994726, recall=1.000000, precision_upper_bound=0.073249, rules=5
Trial 4 (seed 45): best=107.35994726, recall=1.000000, precision_upper_bound=0.073249, rules=5
Trial 5 (seed 46): best=179.731372549, recall=1.000000, precision=0.980392, rules=7

2.3 Experiments

2.3.1 Data

The algorithm trains on strings that are generated from the target grammar through a Python script. The data for each experiment consists of 250 example strings and the number of production rule applications required to generate each string. Each dataset includes only positive examples.

2.3.2 Experiment 1

Experiment 1 aims to discover a semantic equivalent to the target grammar

$$S \rightarrow B \mid C$$
$$B \rightarrow aSb \mid ab$$
$$C \rightarrow aSc \mid ac$$

which generates the language $L(G_1) = \{(a^n w \mid w \in \{a, c\}^n,\ n > 0\}$. Examples of accepted strings are "aabb", "aabc", "aaaabccb", "aaaccc". Experiment 1 has a max string length (x) of 14 and was run with four nonterminals, including the start symbol. The purpose of Experiment 1 is to test the validity of the algorithm with a simple nondeterministic grammar.

2.3.3 Experiment 2

Experiment 2 aims to discover a semantic equivalent to the numerical based target grammar

$$S \rightarrow A \mid B \mid AB$$
$$A \rightarrow aA \mid a$$
$$B \rightarrow bBc \mid bc$$

which generates the language $L(G_2) = \{a^n b^m c^m \mid n, m \geq 0,\ n + m > 0\}$. Examples of accepted strings are "abbbccc", "bbcc", "aaaa", and "aaaaaaaabc". Experiment 2 has a max string length (x) of 30 and was run with seven nonterminals, including the start symbol. The purpose of Experiment 2 is to test whether the algorithm can find numeric limitations in structure.

2.3.4 Experiment 3

Experiment 3 aims to discover a semantic equivalent to the structurally complex target grammar

$$S \rightarrow aSa \mid bSb \mid c$$

which generates the language $L(G_3) = \{wcw^R \mid w \in \{a, b\}^n,\ n > 0\}$. Examples of the accepted strings are "abbbcbbba", "aca", "aabcbaa", and "bacab". Experiment 3 has a max string length (x) of 15 and was run with five nonterminals, including the start symbol. The purpose of Experiment 3 is to test whether the algorithm can find more complex structures.

3. ALGORITHM

3.1 Search Space Reductions

To limit the scope of the search, the search space is constrained. The algorithm will be able to select from the nonterminal start symbol N_1, and a predetermined amount of generic nonterminals N_2, N_3, to N_n. the available terminals will be 'a', 'b', and 'c'. The production rules will be limited to the forms available in Chomsky Normal Form ($A \rightarrow BC$, $A \rightarrow a$).

3.2 Individuals

3.2.1 Representation

Each individual is represented as a 2-dimensional array with each row isomorphic to a production rule in Chomsky Normal Form. The two forms available in the genome are

[$LHS,\ Nonterminal\ 1,\ Nonterminal\ 2$] and [$LHS,\ Terminal,\ \varepsilon$], which are equivalent to the Chomsky Normal Form rules of form $A \to BC$, $A \to a$. Additionally, each individual holds a fitness value.

3.2.2 Individual Creation

Each individual is initialized with a "scaffolding" grammar, which ensures that all production rules are reachable and productive. Scaffolding grammars are covered in more depth in 3.5.

3.2.3 Mutation

Each individual will undergo mutation once per reproduction cycle, following crossover. In mutation, nodes may only mutate to the same type of node, meaning nonterminals may only mutate to nonterminals and terminals to terminals. Each symbol in an individuals' genome has a probability of $1/(length\ of\ the\ genome)$ to mutate. If the grammar can no longer terminate after mutation, then the change is reversed and mutation is run again. After mutation, all useless rules are pruned.

3.2.4 Fitness

The fitness function is evaluated over all provided data. The fitness of an individual is given by a piecewise transform over a core fitness function, with the transform formalized as

$$\tilde{F} = \begin{cases} F, & F < 95, \\ 95, & 95 \leq F \leq 110, \\ F, & F > 110. \end{cases}$$

The fitness is flattened in the range of [95, 110] is the range in which all observed over generalization local optima reside, as per Paper 1. This allows for multi-generation mutation/crossover to take place, enabling the escape of overgeneralization local optima.

The fitness function heavily weights recall through the constant C_1. Recall is the ratio of generated strings that are matched from the dataset divided by the total size of the dataset and is prioritized to encourage specific matching. To prevent a short, generalized grammar that includes incorrect strings in addition to the data, a precision function is added and scaled by constant C_2. In order to build a smoother gradient across the fitness landscape, a normalized Levenshtein distance is added and averaged across the number of all data examples, then scaled by a constant C_3. The Levenshtein difference between two strings is defined by the number of one-character edits required to make the strings match. Lastly, the number of production rules in the candidate grammar is subtracted, scaled by a small constant C_4 to discourage overfitting through hard coded terminal memorization. The function is formalized as

$$F(G) = C_1 \frac{|K|}{|E|} + C_2 \frac{|K|}{|S_{L<x}|} + \frac{C_3}{|E|} \sum_{e \in E} \max_{s \in S_G} \left(1 - \frac{Lev(e,s)}{\max\{|e|, |s|\}}\right) - C_4 |P_G|$$

where

$G = candidate\ grammar$,

$K = set\ of\ string\ matched\ from\ the\ data\ set$,

$S_{L<x} = set\ of\ strings\ generated\ by\ G\ under\ length\ x,$
$E = set\ of\ all\ strings\ in\ the\ data\ set,$
$Lev(e, s) = Levenshtein\ distance\ between\ strings\ e\ and\ s\ ,$
$P_G = number\ of\ production\ rules\ in\ G,$

C_1, C_2, C_3, and C_4 are constants such that $C_1 > C_2 > C_3 > C_4$ and are predetermined before a trial. In this paper, $C_1 = 100$, $C_2 = 80$, $C_3 = 2$, $C_4 = 0.1$. For a given experiment, x is chosen to minimize computation while meeting the string length requirement for 250 data samples to be generated. To make the algorithm more efficient, a fitness of zero is assigned to generated strings that are longer than the corresponding example string.

3.3 Population

3.3.1 Initial Generation

The first generations are initialized with 100 randomly generated valid individuals.

3.3.2 Operating Generations and Reproduction

Each generation undergoes elitism followed by repeated selection, crossover, and mutation. The algorithm utilizes an elitism of size four. The algorithm terminates either after 1000 generations, or after a semantic equivalent to the target grammar is reached.

3.4 Selection

Selection is performed with fitness proportional selection over a sample population of size 10. Each individual is assigned a likelihood to be chosen of

$$P(i \mid S) = \frac{\tilde{F}_i}{\sum_{j \in S} \tilde{F}_j}, \qquad i \in S.$$

This cycle is repeated to select a second parent.

3.5 Reproduction

Each pair of parents results in four offspring. Two offspring are genetic copies and undergo crossover and mutation. The remaining two offspring are genetic copies and undergo only mutation.

3.6 Crossover

After two parents are chosen through selection, they crossover. Crossover involves copying all production rules from both parents to both offspring. First, a “scaffolding” path is set, which guarantees that all non terminals can be reached. The second pass randomly adds one additional production rule per nonterminal to increase the expression of the grammar. If no scaffolding route is possible, crossover fails and two new parents are found through selection. After crossover, all useless rules are pruned.

3.7 Ensuring Grammar Usefulness

For each set of production rules, a valid scaffolding grammar is constructed using a bottom-up approach. Rules that produce terminals are marked as productive, and recursively traced along all dependency paths through the previous rules until at least one path reaches the start symbol. A binary production rule of form $A \rightarrow BC$ is validated once both B and C are demonstrated to be able to reach a productive rule. To prevent infinite recursion, a nonterminal may not be revisited on the same dependency path.

4. RESULTS

4.1 Experiment 1

4.1.1. Data

4.1.2 Analysis

4.2 Experiment 2

4.2.2 Analysis

4.3 Experiment 3

4.3.3 Data

4.3.2 Analysis

5. CONCLUSION

6. APPENDICES

6.1 Experiment 1 Seed 42 Data

6.2 Experiment 2 Seed 42 Data

6.3 Experiment 3 Seed 42 Data